DATA STRUCTURES

UNIT 3: LINKED LIST

POLYNOMIAL CALCULATOR

Polynomial Calculator Using All Types of Linked Lists

MODIFIED DOCUMENTATION

Subject	Data Structures
Unit	Unit 3 – Linked List
Project	Polynomial Calculator
Documentation	Modified
Programming Language	C++

Prepared for Second-Year Engineering – Data Structures Project Documentation

1. Introduction

A polynomial is an expression made up of terms such as $4x^3 + 2x^2 – 5$. This modified project demonstrates polynomial processing through four linked-list representations: singly linked list (SLL), doubly linked list (DLL), circular singly linked list (CSLL), and circular doubly linked list (CDLL). The same polynomial term information is stored in each representation, allowing the project to compare traversal and link-management methods while supporting polynomial operations.

2. Problem Statement

Develop a C++ polynomial calculator that demonstrates the use of all four major linked-list types—SLL, DLL, CSLL, and CDLL. The system should create and traverse polynomial terms using the selected list representation and perform addition, subtraction, multiplication, and result display, while combining like terms and sorting terms by exponent.

3. Objectives

- To represent each polynomial term using a node containing coefficient and exponent.
- To understand dynamic memory allocation and links between nodes.
- To perform polynomial addition and subtraction by combining terms with equal exponents.
- To multiply two polynomials and simplify the resulting expression.
- To display polynomial terms in descending order of exponents.
- To implement and compare SLL, DLL, CSLL, and CDLL representations.
- To understand forward traversal, backward traversal, and circular linking.

4. Data Structure Used

Four linked-list types: SLL has one forward link; DLL has previous and next links; CSLL links the last node back to the first; CDLL links nodes in both directions and connects the last and first nodes circularly. Each node stores coefficient and exponent.

List type	Node links / key feature
SLL	next pointer; last node points to NULL.
DLL	prev and next pointers; supports forward and backward traversal.
CSLL	last node points back to the first node.
CDLL	prev and next pointers; first and last nodes are connected circularly.

5. Algorithm

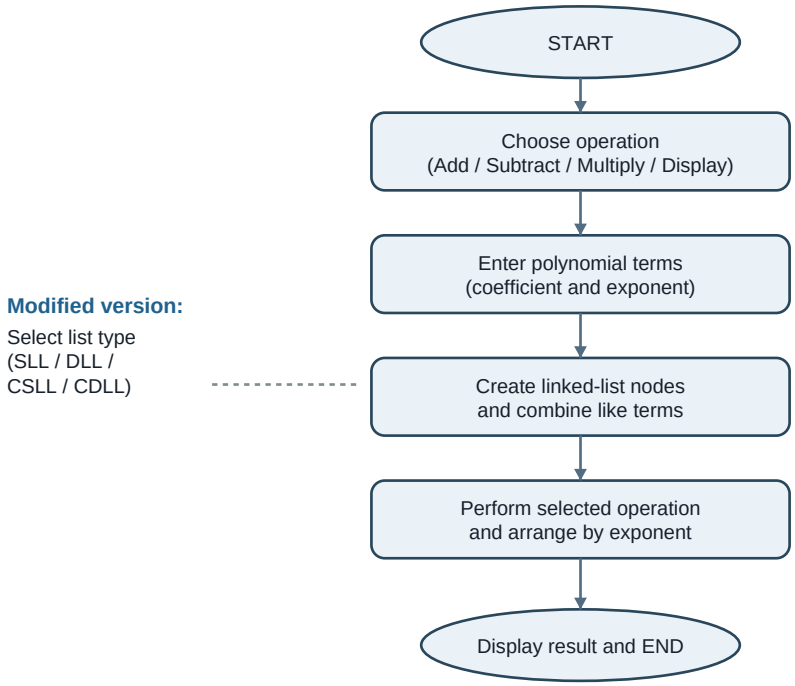
Algorithm: Polynomial Calculator

1. Start the program.
2. Choose the linked-list type (modified version only).
3. Read the terms of Polynomial P and Polynomial Q as coefficient–exponent pairs.
4. Create a node for each term and insert it into the selected linked list.
5. Arrange terms by exponent and combine nodes having equal exponents.
6. Display the menu: Add, Subtract, Multiply, or Display.
7. For addition/subtraction, combine coefficients of terms with equal exponents.
8. For multiplication, multiply every term of P by every term of Q; add exponents and multiply coefficients.

9. Combine like terms in the result and remove zero-coefficient terms.
10. Display the simplified polynomial and stop, or return to the menu.

6. Flowchart

The flowchart below shows the overall processing sequence. In the modified version, list-type selection is included before term creation.



7. Working of the Project

The user first selects SLL, DLL, CSLL, or CDLL, then enters the terms of two polynomials. The program builds the selected representation and uses the appropriate traversal rule: stop at NULL for linear lists, or stop when traversal returns to the starting node for circular lists. Polynomial operations use the same mathematical rules for all list types. Addition and subtraction combine equal exponents; multiplication multiplies coefficients and adds exponents. Finally, like terms are combined and the simplified result is displayed.

8. Programming Language and Concepts

- **Language:** C++.
- **Structures/classes:** Represent polynomial nodes and list operations.
- **Pointers:** Connect nodes and support dynamic list traversal.
- **Dynamic memory allocation:** Create nodes as required.
- **Functions:** Separate input, insertion, display, simplification, and arithmetic operations.
- **Loops and conditional statements:** Traverse lists and match exponents.
- **Arithmetic rules:** For multiplication, (a x■)(b x■) = ab x■■■.
- **List-specific linking:** Manage next, prev, and circular connections correctly.

9. Advantages

- Memory is allocated dynamically according to the number of terms.
- Polynomial size is not restricted to a fixed array length.

- Like terms can be combined to simplify the result.
- The program is modular and can be extended with additional operations.
- Demonstrates differences between four linked-list types in one project.
- DLL and CDLL support backward traversal; circular lists demonstrate cyclic traversal.

10. Applications

- Computer algebra and symbolic mathematical calculations.
- Educational demonstration of linked lists and dynamic memory.
- Polynomial manipulation in engineering and scientific computations.
- Practice in pointer-based data structures and modular C++ programming.

11. Conclusion

The modified project extends the basic polynomial calculator by demonstrating SLL, DLL, CSLL, and CDLL. It highlights how link structure changes traversal and node management, while polynomial arithmetic remains based on the same mathematical rules. The project provides practical understanding of linked-list variations and their implementation in C++.

12. Sample Input and Output

Input	Polynomial P = $3x^2 + 2x + 1$ Polynomial Q = $x + 4$
Addition	P + Q = $3x^2 + 3x + 5$
Subtraction	P – Q = $3x^2 + x – 3$
Multiplication	P × Q = $3x^3 + 14x^2 + 9x + 4$

Note: The sample results are illustrative. The final C++ implementation should validate input, handle empty polynomials, and avoid duplicate or zero-coefficient terms.